06. STARK 증명 생성 과정 분석 (STARK Proof Generation Process Analysis)

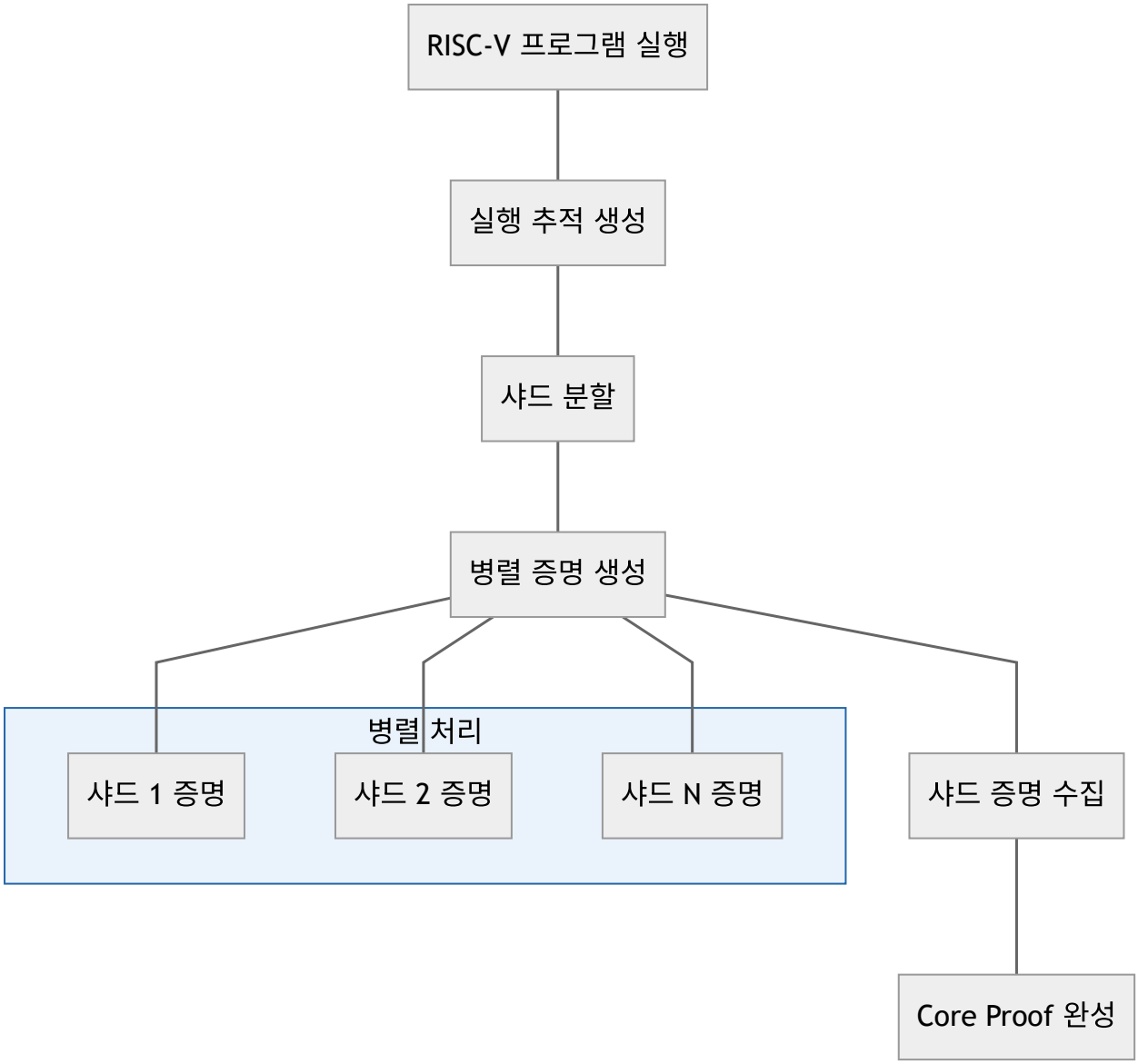
개요

SP1의 STARK 증명 생성은 RISC-V 프로그램의 실행을 영지식으로 증명하는 핵심 과정입니다. 이 문서는 Core Proof 생성 단계, 샤드 분할 및 병렬 처리 방식, 그리고 제약 조건 검증 및 다항식 커밋 과정을 상세히 분석합니다.

1. STARK 증명 생성 전체 흐름

1.1 증명 생성 파이프라인

SP1의 STARK 증명 생성은 다음과 같은 단계로 구성됩니다:



1.2 주요 컴포넌트

- **Executor**: RISC-V 명령어 실행 및 추적 생성
- **Prover**: STARK 증명 생성 엔진
- **Shard**: 실행을 분할한 단위
- **Challenger**: Fiat-Shamir 변환을 위한 챌린지 생성기

2. Core Proof 생성 단계별 분석

2.1 prove_core 함수 구조

```
pub fn prove_core<'a>(
&'a self,
pk_d: &DeviceProvingKey,
program: Program,
stdin: &SP1Stdin,
opts: SP1ProverOpts,
mut context: SP1Context<'a>,
) -> Result<SP1CoreProof, SP1CoreProverError>
```

주요 단계:

1. **컨텍스트 설정**: 서브프루프 검증기 설정
2. **병렬 스레드 생성**: 증명 생성과 재귀 프로그램 컴파일 동시 실행
3. **증명 스트리밍**: `prove_core_stream` 함수를 통한 실제 증명 생성
4. **결과 수집**: 생성된 샤드 증명들을 하나의 Core Proof로 통합

2.2 prove_core_stream 함수 분석

이 함수는 실제 STARK 증명 생성의 핵심 로직을 담당합니다:

주요 구성 요소:

1. **Runtime 설정**

```
let mut runtime = Box::new(Executor::with_context(program.clone(), opts, conte
runtime.maximal_shapes = shape_config.map(|config| {
config.maximal_core_shapes(opts.shard_size.ilog2() as usize).into_iter().collect
```

```
});
```

2. **체크포인트 생성기 스레드**

- 프로그램 실행을 청크 단위로 분할
- 각 청크의 실행 상태를 체크포인트로 저장
- 임시 파일을 통한 메모리 효율적 관리

3. **Phase 2 레코드 생성기 스레드**

- 체크포인트로부터 실행 레코드 복원
- 추적 데이터 생성 및 형태 고정
- 의존성 생성 및 공개 값 업데이트

4. **Phase 2 증명 생성기 스레드**

- 실행 레코드를 기반으로 STARK 증명 생성
- 병렬 처리를 통한 성능 최적화

3. 샤드 분할 및 병렬 처리 방식

3.1 샤드 분할 전략

SP1은 대용량 프로그램의 효율적 처리를 위해 실행을 여러 샤드로 분할합니다:

분할 기준:

- **사이클 수**: `opts.shard_size` 로 지정된 최대 사이클 수
- **메모리 사용량**: 메모리 초기화/종료 이벤트 수
- **의존성**: 샤드 간 데이터 의존성 고려

샤드 구조:

```rust
pub struct ExecutionRecord {
pub shard: u32,
pub public_values: PublicValues<u32, u32>,
pub cpu_events: Vec<CpuEvent>,
pub memory_initialize_events: Vec<MemoryInitializeEvent>,
pub memory_finalize_events: Vec<MemoryFinalizeEvent>,
// ... 기타 이벤트들
}
```

3.2 병렬 처리 아키텍처

SP1은 다단계 병렬 처리를 통해 성능을 최적화합니다:

스레드 구조:

1. **체크포인트 생성기**: 단일 스레드로 순차 실행
2. **추적 생성기**: `opts.trace_gen_workers` 개수만큼 병렬 실행
3. **증명 생성기**: 단일 스레드로 배치 처리

동기화 메커니즘:

```rust
let p2_record_gen_sync = Arc::new(TurnBasedSync::new());
let p2_trace_gen_sync = Arc::new(TurnBasedSync::new());
```

- **TurnBasedSync**: 순서 보장이 필요한 작업의 동기화
- **Channel**: 스레드 간 데이터 전달
- **Arc<Mutex<>>**: 공유 상태 관리

3.3 메모리 최적화

임시 파일 사용:

```
let mut checkpoint_file = tempfile::tempfile().map_err(SP1CoreProverError::IoE
checkpoint.save(&mut checkpoint_file).map_err(SP1CoreProverError::IoError)?;
```

청킹 전략:

```
let chunked_records = chunk_vec(records, opts.shard_batch_size);
let chunked_main_traces = chunk_vec(main_traces, opts.shard_batch_size);
```

4. 제약 조건 검증 및 다항식 커밋 과정

4.1 STARK 증명 구조

각 샤드에 대한 STARK 증명은 다음 구조를 가집니다:

```
pub struct ShardProof<SC: StarkGenericConfig> {
pub commitment: ShardCommitment<Com<SC>>,
pub opened_values: ShardOpenedValues<Val<SC>, Challenge<SC>>,
pub opening_proof: OpeningProof<SC>,
pub chip_ordering: BTreeMap<String, usize>,
pub public_values: Vec<Val<SC>>,
}
```

4.2 증명 생성 과정

1단계: 메인 추적 생성

```
main_traces = records
.par_iter()
.map(|record| prover.generate_traces(record))
.collect::<Vec<_>>();
```

2단계: 커밋 단계

```
let main_data = tracing::debug_span!("commit", shard)
.in_scope(|| prover.commit(&record, main_traces));
```

- 실행 추적을 다항식으로 변환
- Merkle 트리를 통한 커밋 생성
- 공개 값 설정

3단계: 오프닝 단계

```
let proof = tracing::debug_span!("opening", shard).in_scope(|| {
prover.open(pk, main_data, &mut challenger.clone()).unwrap()
});
```

- 챌린지 포인트에서의 다항식 평가
- FRI 프로토콜을 통한 오프닝 증명 생성
- 최종 샤드 증명 구성

4.3 제약 조건 시스템

SP1은 AIR (Algebraic Intermediate Representation)를 사용하여 제약 조건을 정의합니다:

칩별 제약 조건:

- **CPU 칩**: RISC-V 명령어 실행 제약
- **메모리 칩**: 메모리 일관성 제약
- **시스템 콜 칩**: 시스템 콜 처리 제약

글로벌 제약 조건:

- **누적 합**: 샤드 간 일관성 보장
- **공개 값**: 입출력 데이터 검증
- **상태 전이**: 샤드 간 상태 연속성

4.4 챌린지 생성 및 Fiat-Shamir 변환

```
let mut challenger = prover.config().challenger();

pk.observe_into(&mut challenger);
```

챌린지 생성 과정:

1. **초기 시드**: 증명 키 관찰
2. **커밋 관찰**: 각 라운드의 커밋 값 흡수
3. **챌린지 추출**: 무작위성 생성
4. **다음 라운드**: 생성된 챌린지로 다음 단계 진행

5. 성능 최적화 기법

5.1 병렬 처리 최적화

워커 스레드 수 조정:

```
for _ in 0..opts.trace_gen_workers {
// 추적 생성 워커 생성
}
```

배치 처리:

```
let chunked_records = chunk_vec(records, opts.shard_batch_size);
```

5.2 메모리 관리

지연 삭제:

```
rayon::spawn(move || {
drop(record);
});
```

채널 용량 조정:

```
sync_channel::<(usize, File, bool, u64)>(opts.checkpoints_channel_capacity);
```

5.3 형태 고정 (Shape Fixing)

```
if let Some(shape_config) = shape_config {
for record in records.iter_mut() {
```

```
shape_config.fix_shape(record).unwrap();
}
}
```

- 증명 크기 예측 가능성 향상
- 메모리 사용량 최적화
- 증명 생성 시간 단축

6. 오류 처리 및 디버깅

6.1 오류 타입

```
pub enum SP1CoreProverError {
ExecutionError(ExecutionError),
IoError(io::Error),
// ... 기타 오류 타입들
}
```

6.2 디버깅 기능

제약 조건 디버깅:

```
#[cfg(feature = "debug")]
{
let all_records = all_records_rx.iter().flatten().collect::<Vec<_>>();
```

```
let mut challenger = prover.machine().config().challenger();

let pk_host = prover.pk_to_host(pk);

prover.machine().debug_constraints(&pk_host, all_records, &mut challenger);

}
```

성능 모니터링:

```
tracing::debug!(

"proving shard {shard} took {} ns. shape: {:?}",

elapsed.as_nanos(),

debug_shapes

);
```

7. 결론

SP1의 STARK 증명 생성 과정은 다음과 같은 특징을 가집니다:

1. **확장성**: 샤드 분할을 통한 대용량 프로그램 지원
2. **효율성**: 다단계 병렬 처리를 통한 성능 최적화
3. **안정성**: 강력한 오류 처리 및 디버깅 기능
4. **유연성**: 설정 가능한 매개변수를 통한 최적화

이러한 설계를 통해 SP1은 실용적인 영지식 증명 시스템으로서의 성능과 안정성을 동시에 확보하고 있습니다.